

MICR_ME601 Microcontrollers

Hardware Interfaces & Control Part 1

Dr John Wade

Email: john.wade@atu.ie



Overview

- Interfacing with mechanical switches
 - Problems
- Floating pins
 - Why its bad and how to avoid
- Interfacing to Matrix Keypads
- Interfacing to LEDs
 - Calculations Needed
- Interfacing with 7 Segment Displays
 - BCD and Multiplexing
- Interfacing with DC Motors
 - Relays, H-bridges
- Interfacing to LCD Displays



Interfacing to Switches

Interfacing to Switches

One of the most common types of input

- In most systems where you use a microcontroller you need to sense the outside world.
- A lot of this sensing is done by switches activated at specific pressures, temperatures, humidity, etc. or a push button momentary switch activated by the user.
- **Examples of real-world switch use:**

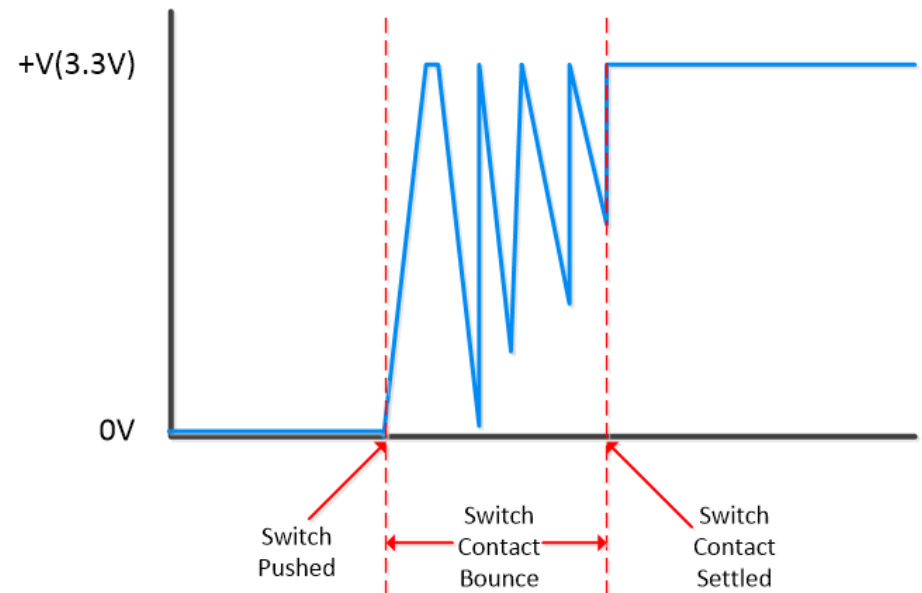
An automotive speed control - typically a switch on the brake (and clutch) pedal that disengages the speed control when it is pressed.

Washing machine door release – pressure switch that only allows the washing machine door to open when there is no water in the drum.

Mechanical Switch Problem

Bouncing

- When a switch is switched on or off it is very, very noisy with multiple spikes in voltage as the mechanical contacts open and close rapidly.
- If not correctly processed, these spikes could be seen as multiple switch closures (openings).
- **Solution** – Debounce the switch.

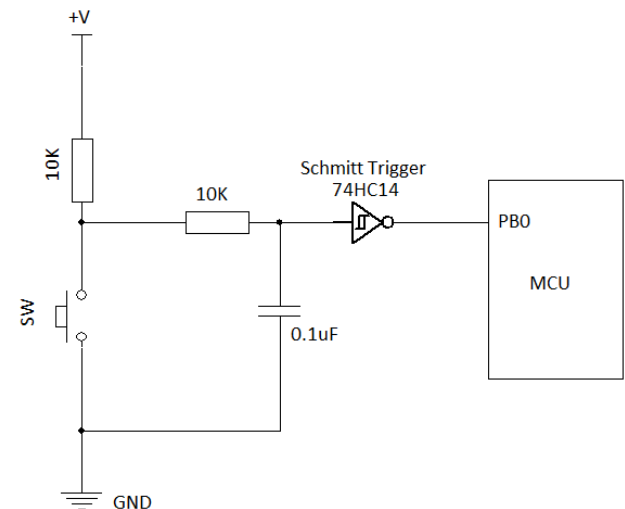
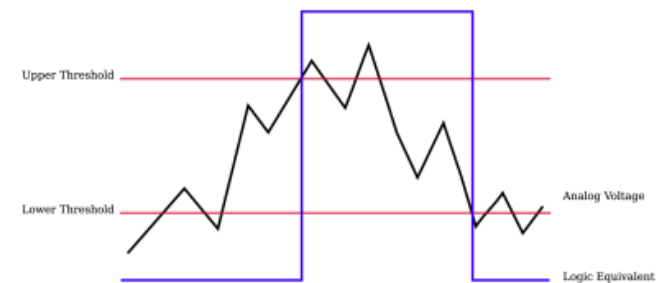


Switch Bounce

Debounce - Hardware

Schmitt trigger - Hysteresis

- Hysteresis: instead of one voltage threshold, right in the middle, there are two.
 - one-third and two-thirds of the maximum logic voltage are common.
- It filters out the “noise” of the switch contact bounce.
- A Schmitt Trigger (74HC14 chip) can provide the necessary hysteresis for a mechanical switch.
- When the mechanical switch is pressed the Schmitt trigger will not output a logic 1 until the input voltage to the chip is greater than the upper threshold.
- Once the upper threshold has been passed, the chip will not output a logic 0 until the input voltage to the chip is lower than the lower threshold.



Debounce - Software

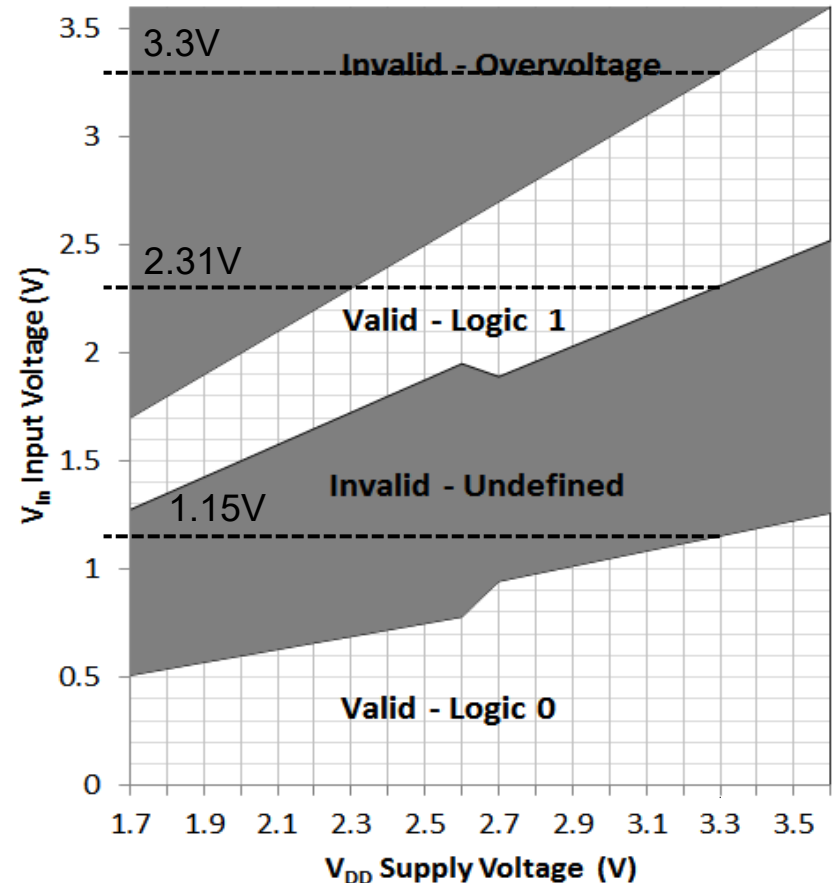
- For non time-critical applications, switch debounce is generally performed in software
- When the software checks if the switch is pressed:
 - If the switch is pressed (e.g. input = 3.3V):
 - wait for a period of time (usually 10ms-100ms)
 - Then check to see if the switch is still pressed.
 - If it is, we can take it that the bouncing has settled and the switch press can be passed to the rest of the software
- **Limitations:** This method uses a blocking delay function to wait for the contact to settle.
 - Not all switches require the same debounce time –
It's a guessing game!!

A note on connection to input pins

- All input pins on the chip **MUST** be connected to an input signal, or connected to V_{DD} or GND as appropriate.
- Digital inputs are interpreted based on their voltage level
 - Normally:
 - $1 = V_{DD}$
 - $0 = 0v$ (GND)

What is 0 and 1?

- Normally:
 - $1 = V_{DD}$
 - $0 = 0V$ (GND)
- In reality 1 and 0 are defined as a range of voltages.
- If $V_{DD} = 3.3V$
 - $1 = 2.31V - 3.3V$
 - $0 = 0V - 1.15V$
- No input pin can be left floating. Why?
 - A floating pin will have an indeterminate voltage level due to noise and may trigger a false 1 or 0 at the input.



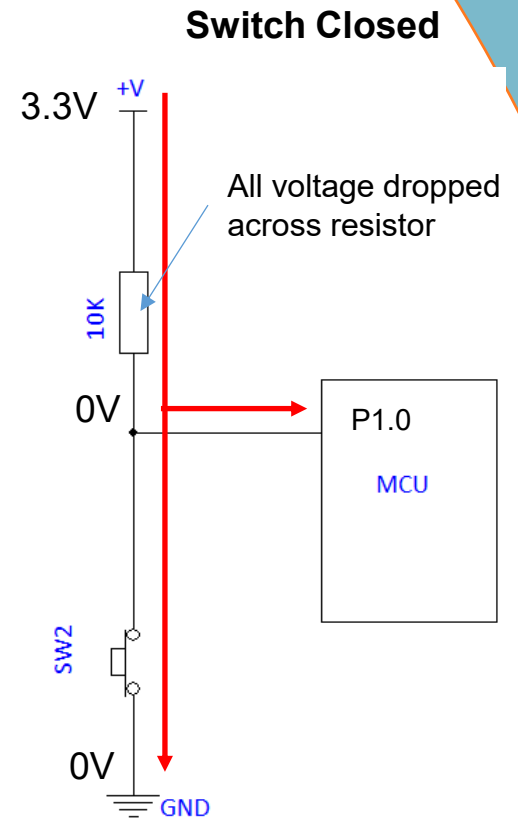
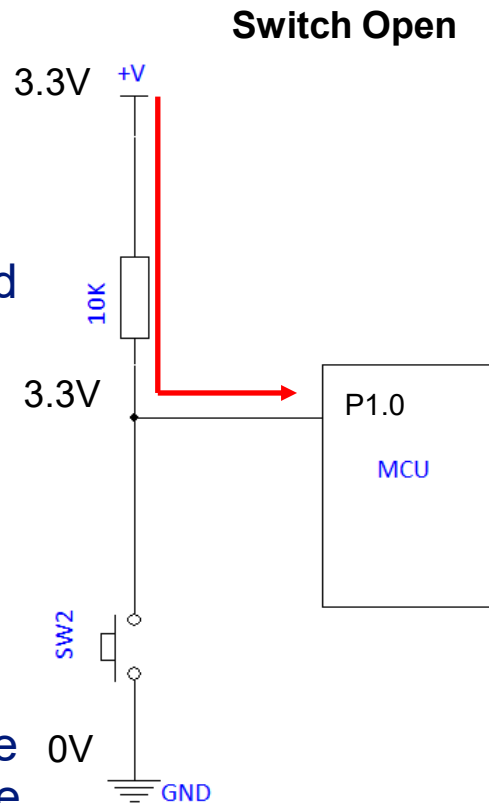
Pull Up Resistors

Ensuring Pins are always connected to V_{DD} when switch is open

- Since MCU draws very little current, when the switch is open there is very little voltage drop across the resistor and the MCU input pin is held at V_{DD} .

- When the switch is closed there is a much larger current flow from V_{DD} to GND (0V) and all the voltage is dropped across the resistor. Therefore, the side of the resistor connected to the MCU is held at 0V

- Typical value 10K Ω .



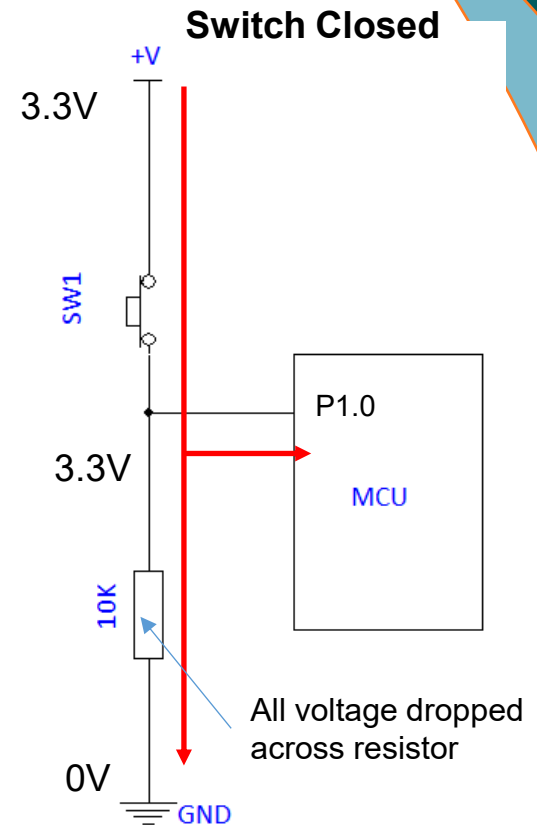
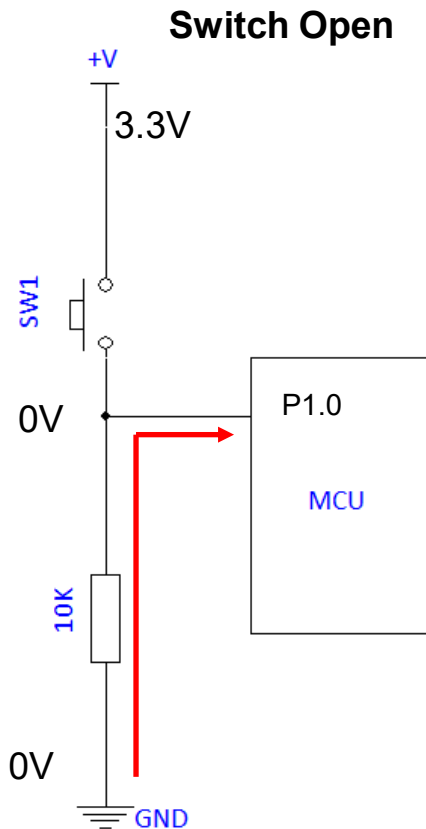
Pull Down Resistors

Ensuring Pins are always connected to 0V when switch is open

- Works in the same way as pull up resistor but is connected to ground instead when the switch is open.

- When the switch is closed, again, all the voltage is dropped across the resistor. Therefore the side of the resistor connected to the MCU is held at V_{DD} .

- Typical value 10K Ω .



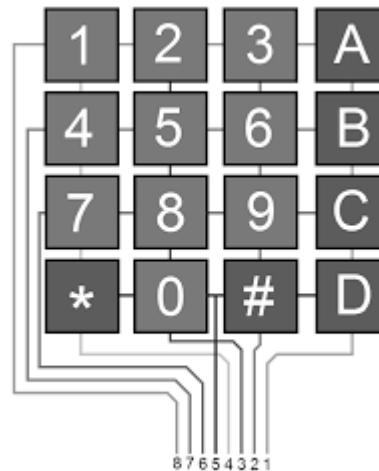
Interfacing to Matrix Keypads

Interfacing with Matrix Keypads

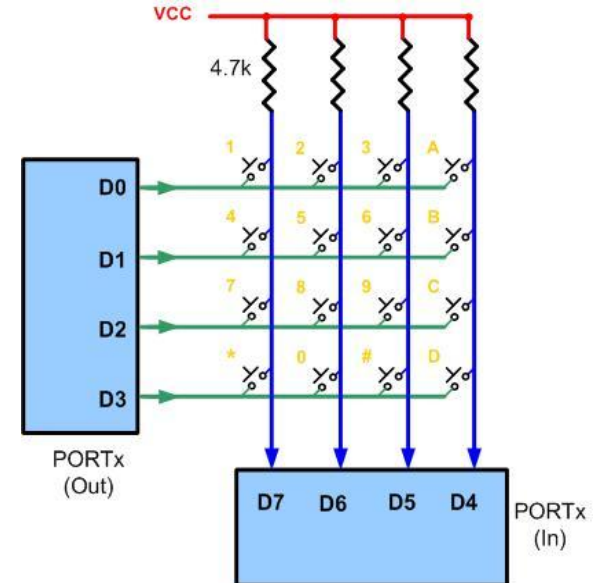
- To reduce MCU I/O pin usage, Keyboards or keypads are organised in a matrix of rows and columns.
- Come in a large range of sizes:
 - 2x2 (4) keys to 8x8 keys (64)
 - So two 8 bit ports is all that is needed for 8x8 keypad



4x4 Keypad



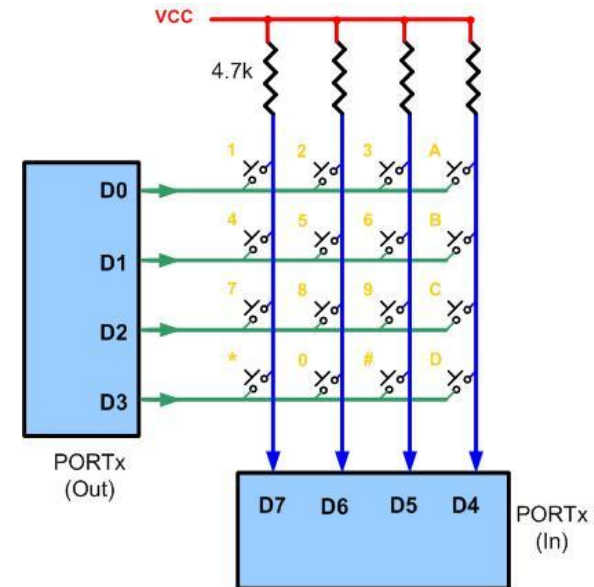
4x4 Keypad Layout



4x4 Keypad Connections

Interfacing with Matrix Keypads

- When a key is pressed a row and column make contact
 - Rows are generally connected to MCU output port pins
 - Cols are generally connected to MCU input port pins
 - Note: all input ports have pull up resistors connected:- generally 4.7K – 10K
- By scanning the inputs and outputs in software a keypress can be detected and decoded, i.e. which key.
- No Key Pressed:**
 - Reading the input pins will yield all 1's
- Key Pressed:**
 - All Outputs are driven low
 - If key pressed the col of that key will read low



4x4 Keypad Connections

Interfacing with Matrix Keypads

Key Press Detection

- **Detect key press steps:**

1. MCU drives all Rows Low i.e. Ground

- (Note: Outputs are in Open Drain Configuration)

2. Reads all the Cols:

- If Data read from Cols D7-D4 is = 1111

- No Key Pressed
- Process continues

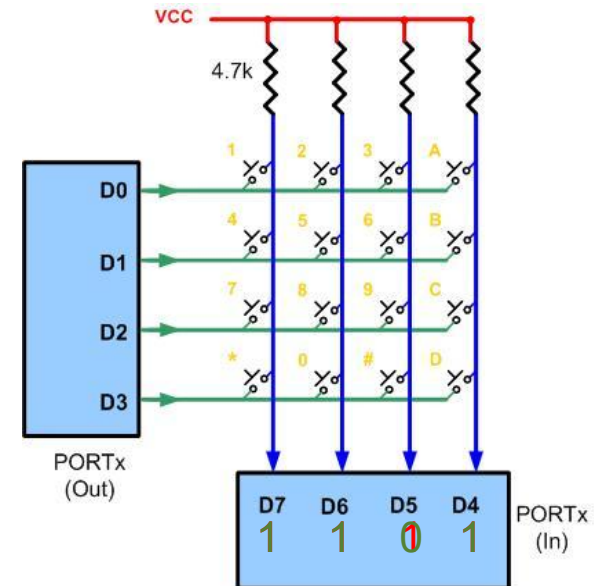
- If Data read from Cols D7-D4 is not = 1111

- Key Pressed
- Example:

D7-D4 = 1101

Means a key in Col 3 Pressed

- Identify Key
- Process Continues

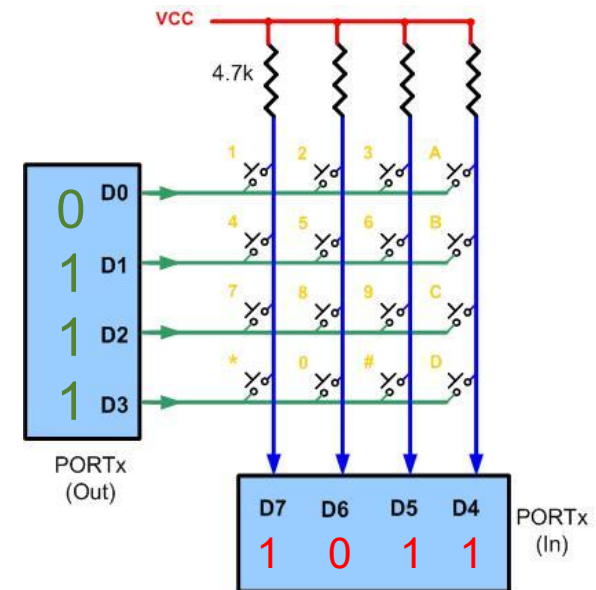


4x4 Keypad Connections

Interfacing with Matrix Keypads

Key Identification

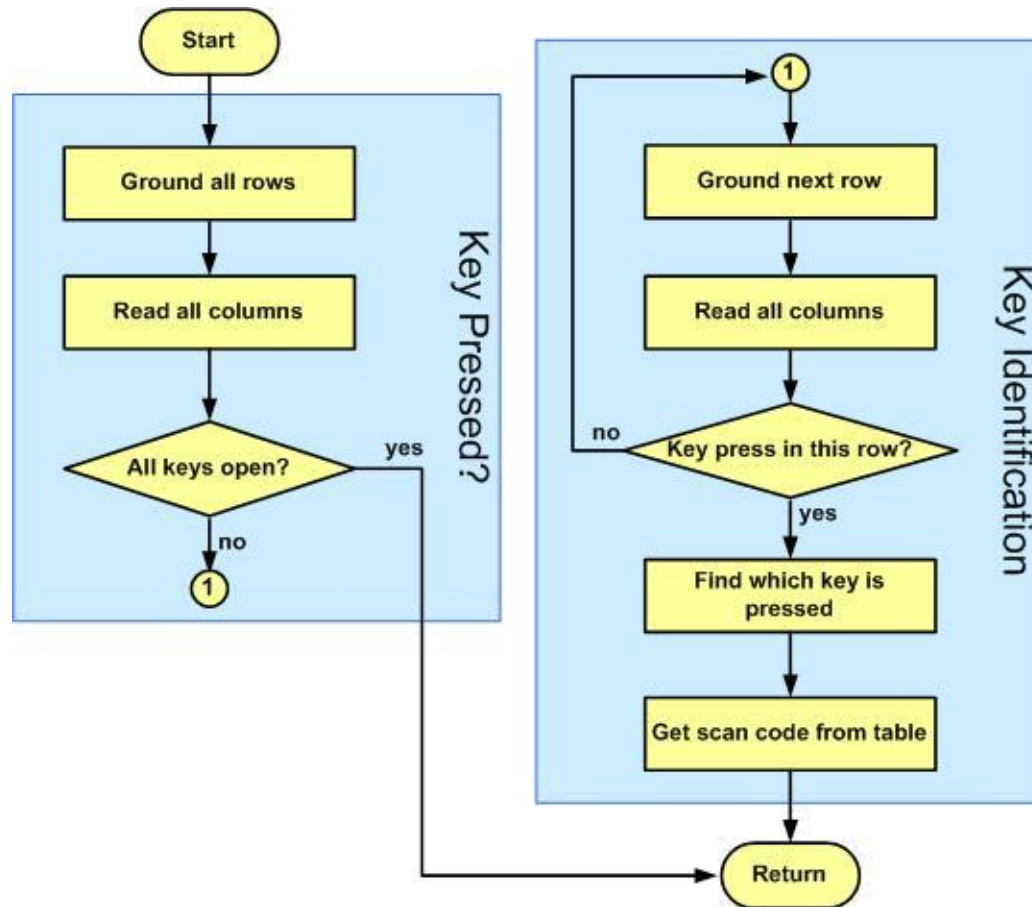
- **Key Identification steps:**
 1. Drive 1st Row low
 2. Read Cols:
 - If Data Read is all 1's, No Key pressed
 - Drive next row Low
 - If Data Read is not all 1's
 - Key pressed and identified
 - Return
- Example: D3-D0 (Rows) = **1110** and D7-D4 (Cols) = **1011**
 - The Row belongs to D0 and the Col belongs to D6:
 - Number 2 Key pressed



4x4 Keypad Connections

Interfacing with Matrix Keypads

Key Press and Identification Flow Chart

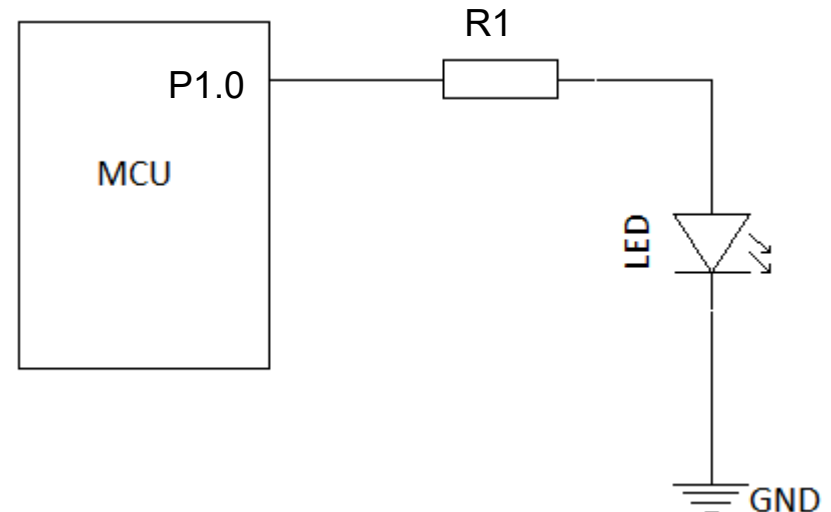


Interfacing to LEDs

Interfacing to Light Emitting Diodes (LEDs)

Positive logic

- LEDs are often used as output devices.
 - e.g. status indication
- Usually, a positive logic circuit for driving an LED is used
 - MCU output is 0 – LED off
 - MCU output is 1 – LED on

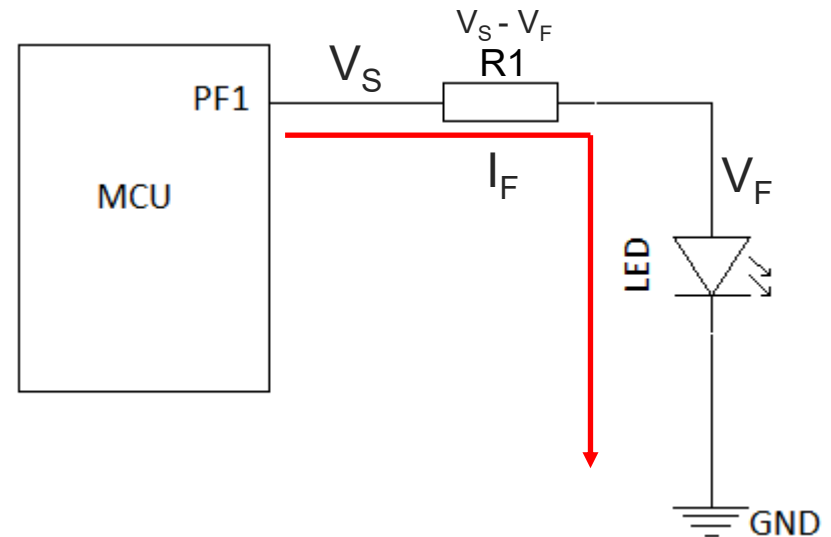


Calculating the limit resistor (R1)

- All LEDs have a Forward voltage (V_F). This is the max voltage that should be applied to the LED or it will burn out.
- They also have a maximum forward current that can flow through them (I_F).
- If we know the voltage supplied from a source or MCU (V_S), V_F and I_F of the LED, then we can work out the value of R1 which will ensure the safe operation of the LED.

- To ensure only V_F is applied to the LED we must ensure that there is a voltage drop across the resistor of $V_S - V_F$
- Using ohms law ($V = I \cdot R$) we can rearrange for R ($R = V/I$)

- Therefore $R1 = \frac{V_S - V_F}{I_F}$



Calculating the limit resistor (R1)

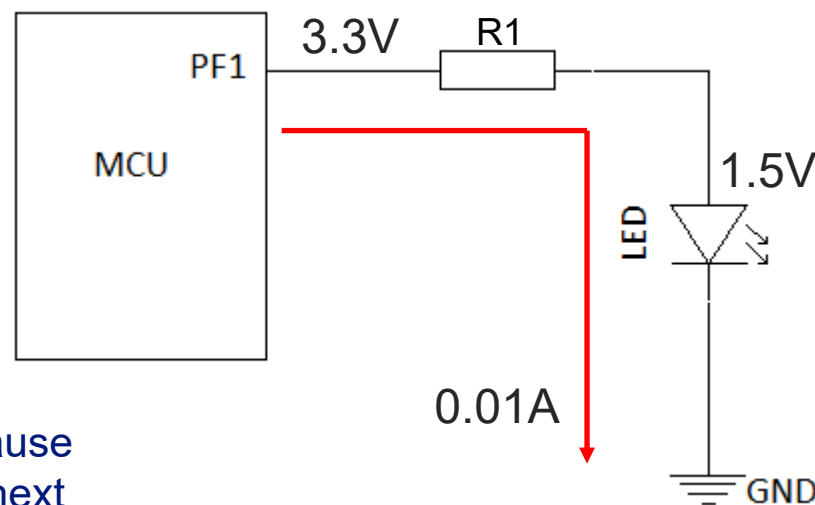
EXAMPLE

- Below shows a microcontroller connected to an LED. If we assume the LED has a Forward Voltage (V_F) = 1.5V and a Forward Current (I_F) = 0.008A, and the microcontroller provides a source voltage (V_S) = 3.3V, calculate the resistance of R1 to ensure the LED does not get damaged when the microcontroller turns it on?

$$R1 = \frac{V_S - V_F}{I_F}$$

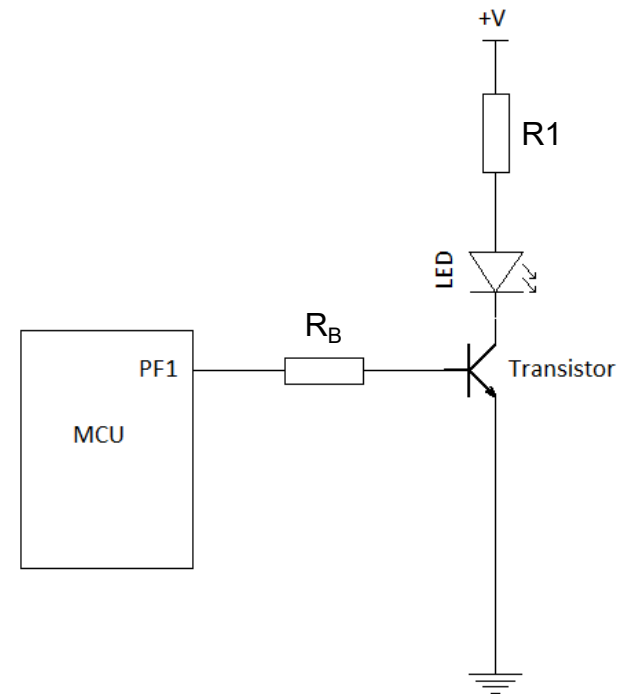
$$\therefore R1 = \frac{3.3V - 1.5V}{0.008A} = 225\Omega$$

If we cannot get a 225 ohm resistor (because they do not exist) we always choose the next highest i.e. 240 ohm



What if LED current is too high for MCU?

- If the LED forward current is too high for the MCU to supply then we can use the following circuit:
- When the output of the MCU is 0 the transistor will be switched off and act as a high impedance (open circuit) and the LED is off.
- When the output of the MCU is 1 the Transistor will switch on and current will flow through it and complete the circuit along the LED to light.
- So how do we calculate R_B and R_1 ?



- We use the transistor as a switch.

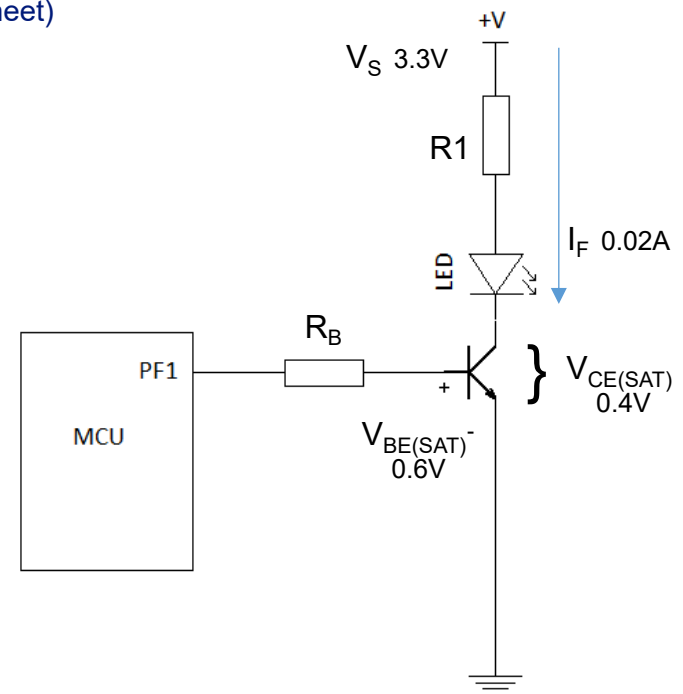
What if LED current is too high for MCU?

Example

- The LED in the circuit shown is to be interfaced to PF1 using a low power general purpose NPN transistor with a $V_{CE(SAT)}$ of 0.4V, $V_{BE(SAT)}$ of 0.6V and a H_{FE} value of 100. V_S is 3.3V, LED $V_F = 1.5V$ and LED $I_F = 0.02A$

(Note: You can find V_{CE} , V_{BE} and H_{FE} for any transistor in it's data sheet)

- $R1 = \frac{V_S - V_F - V_{CE}}{I_F} = \frac{3.3V - 1.5V - 0.4V}{0.02A} = 70\Omega$
- next value up = 75Ω
- $R_B = \frac{V_S - V_{BE}}{I_F / H_{FE}} = \frac{3.3V - 0.6V}{0.02A / 100} = 27.5K\Omega$
- To ensure there is enough current flowing into the base of the transistor we always choose the next lowest value i.e. $22K\Omega$



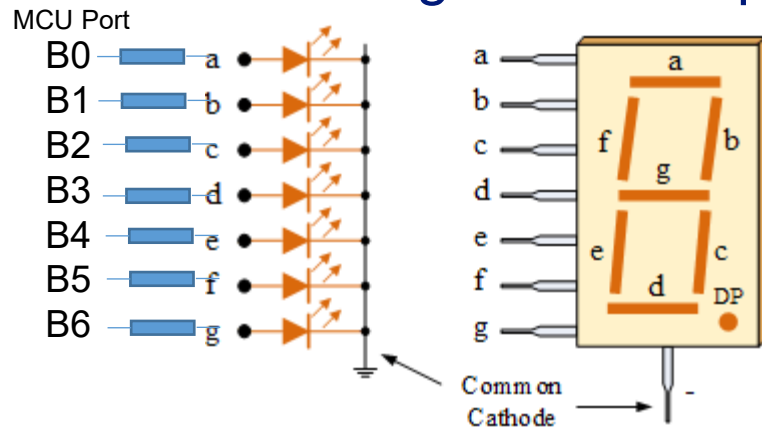
Interfacing to 7 Segment Display

Interfacing to 7 Segment Display

- Often used to display BCD numbers (0 through 9) and a few alphabet characters.
- A group of eight LEDs physically mounted in the shape of the number eight plus a decimal point.
- Each LED is called a **segment** and labeled as 'a' through 'g'.

7 Segment Displays

- Each segment of display is a LED

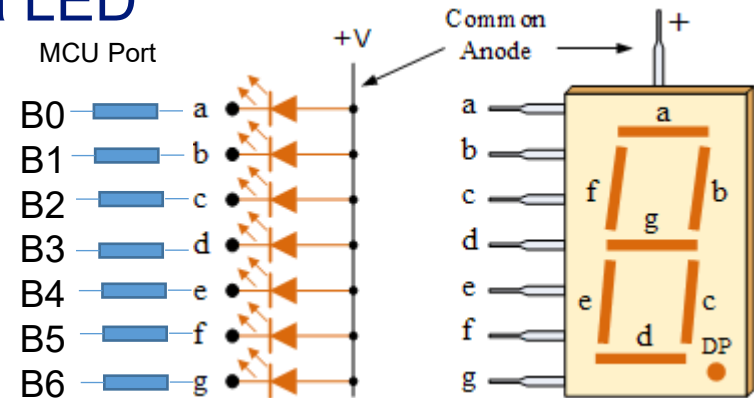


In a common cathode seven-segment LED
All cathodes are connected together to ground and the anodes are connected to data lines

Logic 1 turns on a segment.

Example: To display digit 1, all segments except b and c should be off.

Byte 00000110 = 0x06 will display digit 1



In a common anode seven-segment LED
All anodes are connected together to a power supply and cathodes are connected to data lines

Logic 0 turns on a segment.

Example: To display digit 1, all segments except b and c should be off.

Byte 11111001 = 0XF9 will display digit 1

Binary Coded Decimal

Displaying digits 0 to 9 on a 7 Segment Display

Decimal Digit	Input lines				Output lines							Display pattern
	A	B	C	D	a	b	c	d	e	f	g	
0	0	0	0	0	1	1	1	1	1	1	0	
1	0	0	0	1	0	1	1	0	0	0	0	
2	0	0	1	0	1	1	0	1	1	0	1	
3	0	0	1	1	1	1	1	1	0	0	1	
4	0	1	0	0	0	1	1	0	0	1	1	
5	0	1	0	1	1	0	1	1	0	1	1	
6	0	1	1	0	1	0	1	1	1	1	1	
7	0	1	1	1	1	1	1	0	0	0	0	
8	1	0	0	0	1	1	1	1	1	1	1	
9	1	0	0	1	1	1	1	1	0	1	1	

Driving more than on 7 Segment Display

Multiplexing

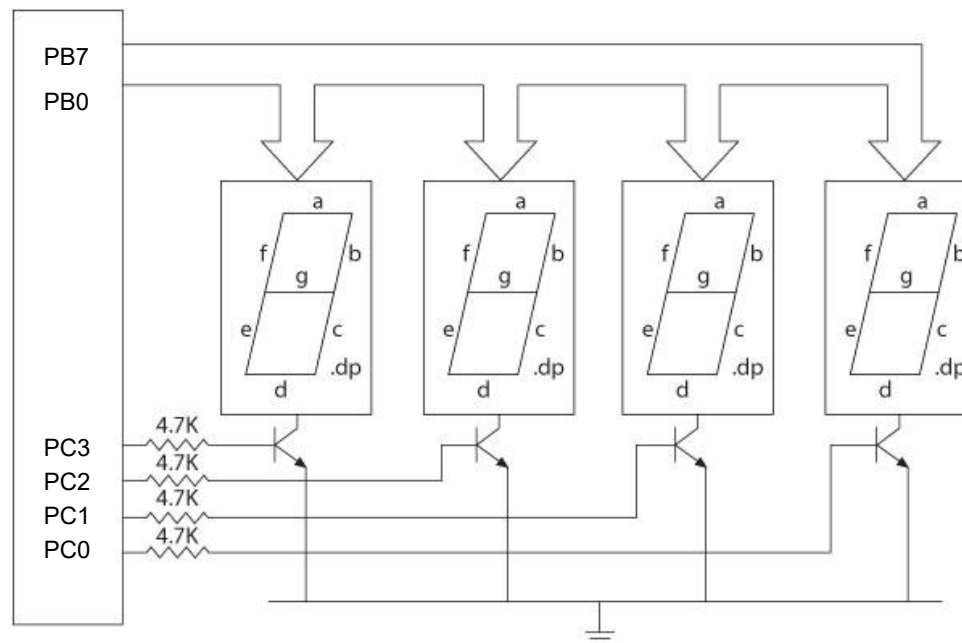
- With just 1 extra pin for each extra display we can turn on each display by using a transistor to switch on the common anode or the common cathode. (There is no need for 7 or 8 extra lines for each display)

First: output desired digit pattern on PB[7:0] and then switch on the corresponding 7 segment display for a short period of time (around 60ms) using another output port (PC0)

Next: Turn off the first display (PC0) and output new digit pattern on PB[7:0] then switch on the next digit via the transistor (PC2) for 60ms.

Continue these steps for the rest of the digits.

Since our eyes cannot detect these fast changes it looks like all the displays are on at the same time.



Common Cathode Display

Interfacing to DC Motors

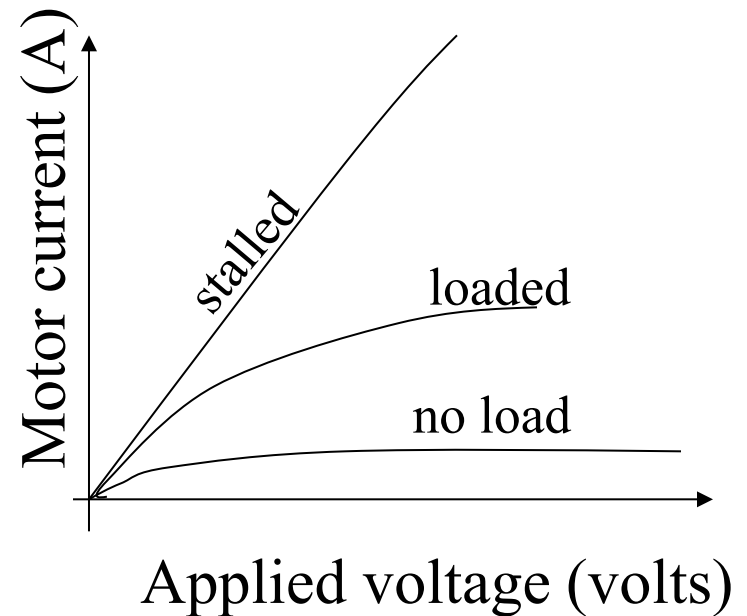
DC Motors

- Important in LOTS of applications
 - cameras, drives, elevators, trains, robots...
- Many types, but all work similarly:
 - Apply voltage across + and – leads, electrical energy is converted to mechanical energy.
 - For some range of voltage, the torque of the motor shaft is proportional to value of voltage.



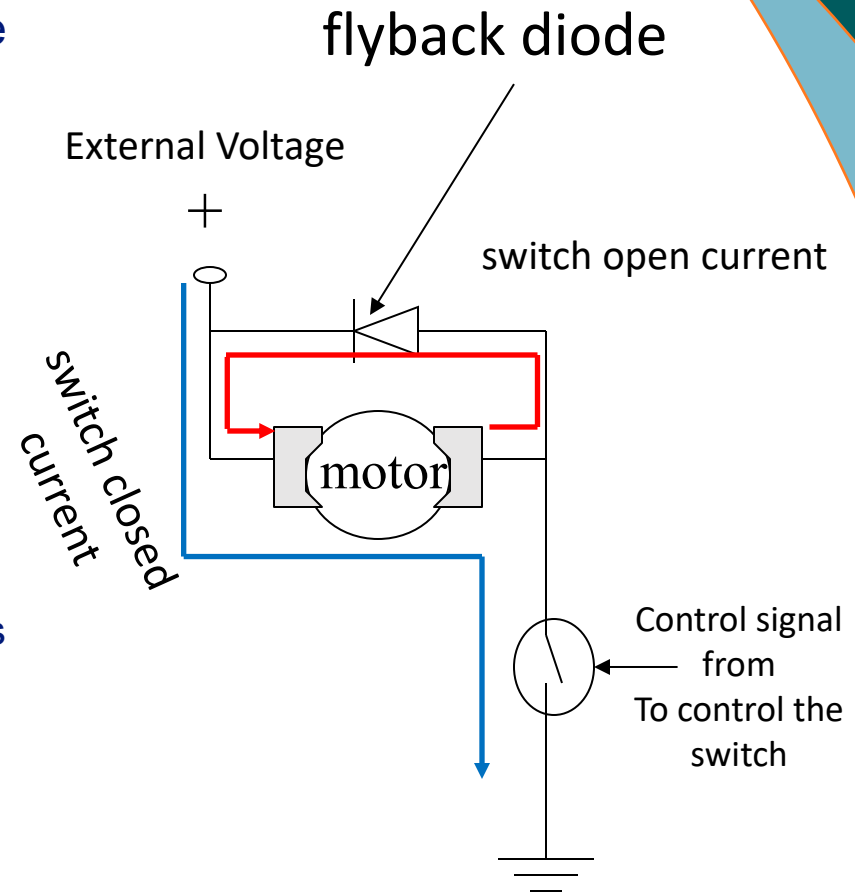
DC Motors

- Current required by motor depends on how it is loaded.
- Current is almost always more than the MCU can provide.
- Need an interface circuit between MCU and motor.



Interfacing to DC Motors

- Basic idea is to use a switch of some kind to isolate current in MCU from motor current.
- The switch can be a Transistor or Relay.
- Motor is an inductor though, so it stores current.
 - Flyback diode used to route current away from switch when switch opens to avoid damage to switch.



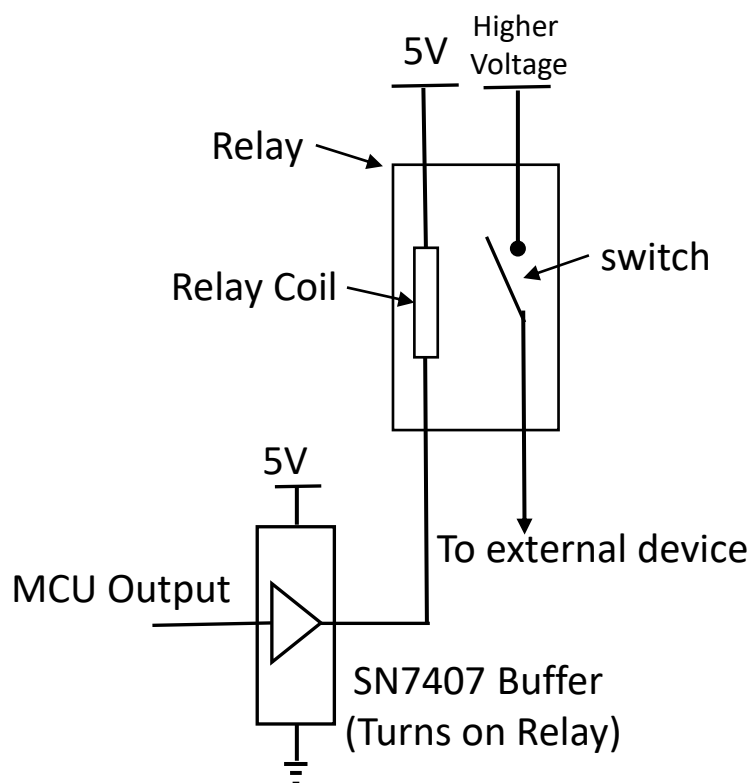
Relays

- Example: An MCU could not turn on a 240VAC light because:
 - A. as the this requires much more current than the MCU can provide and
 - B. the voltage of the MCU is DC and only 3.3V. If the MCU came into contact with 240VAC **it would be destroyed!!**
- Relays are switched on by applying a voltage to the electromagnetic coil. When on the electromagnet pulls on a mechanical switch and makes the switch. When off the switch opens again.
- Relays can also be used to switch on/off (drive) devices which have a very high current draw which the MCU cannot supply.
- It is often used as part of the drive circuit to isolate the MCU from higher voltages that are required by the external components.

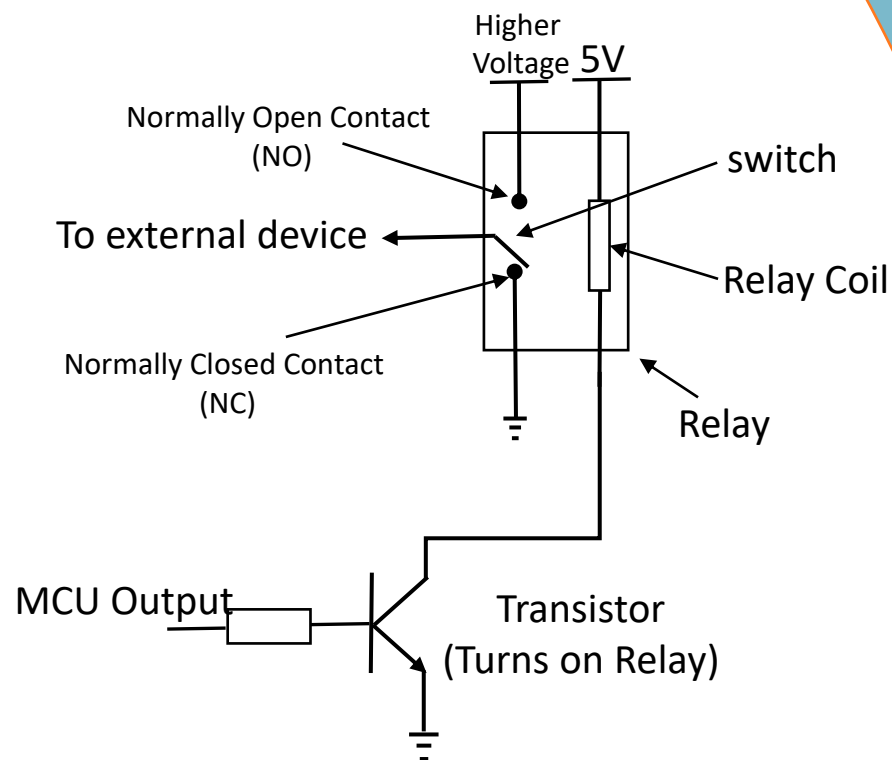
Relays

Two very common types

Single Pole Single Throw (SPST)



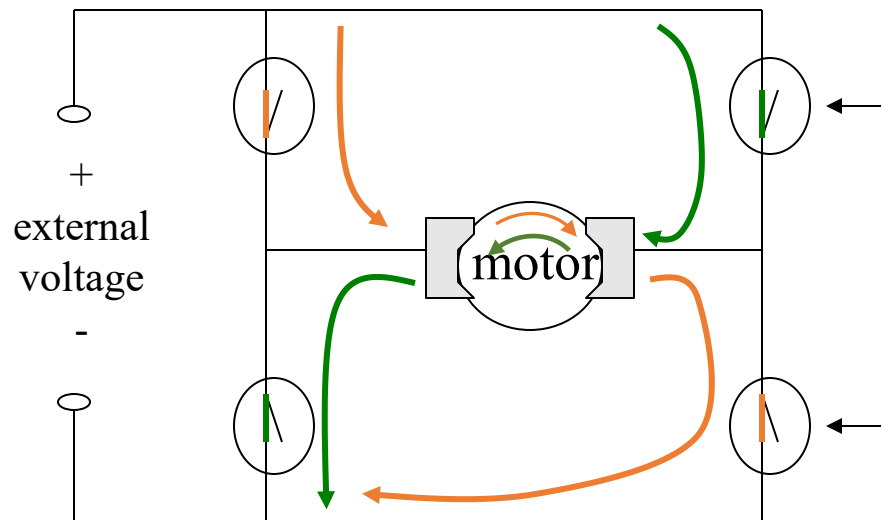
Single Pole Double Throw (SPDT)



H-Bridge

Controlling the direction of a DC Motor

- H-Bridge – circuit topology that allows bi-directional control of motor.



- Each switch controlled by an output of the MCU.
- Switches implemented by relays, solid-state switches, or transistors

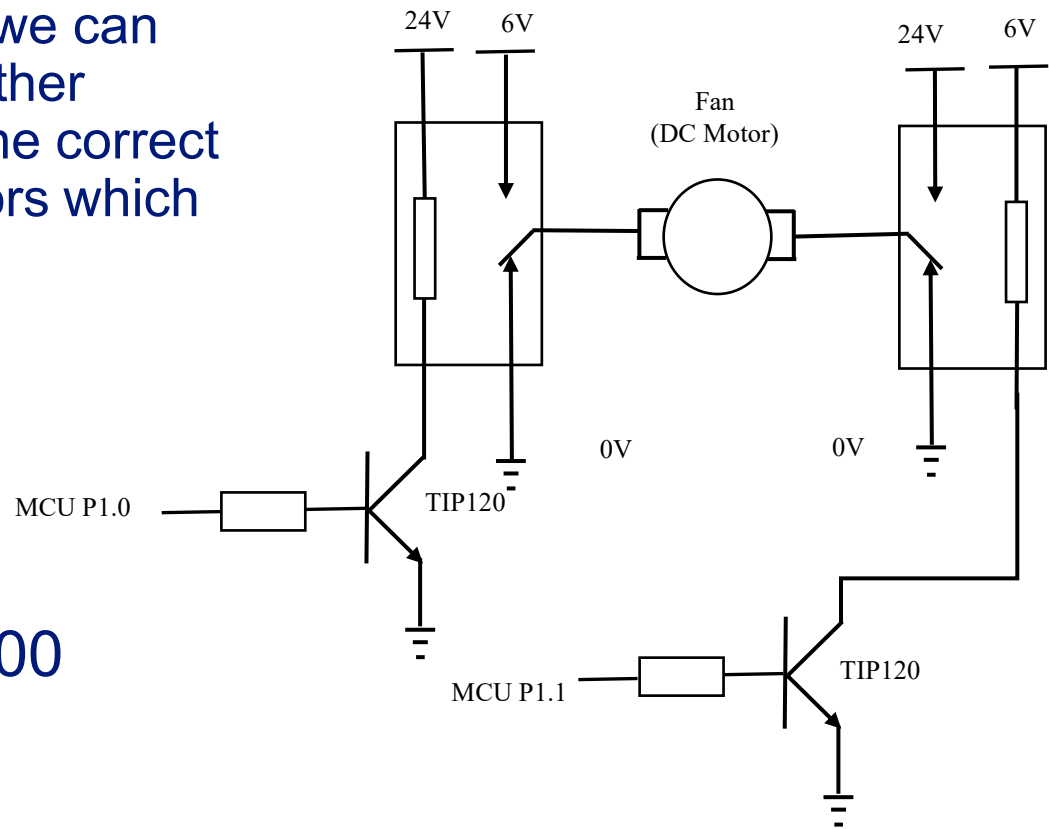
H-Bridge Relay Circuit

OFF

- Using SPDT relays we can drive the motor in either direction by giving the correct code to the transistors which switch on the relay.

Signals:

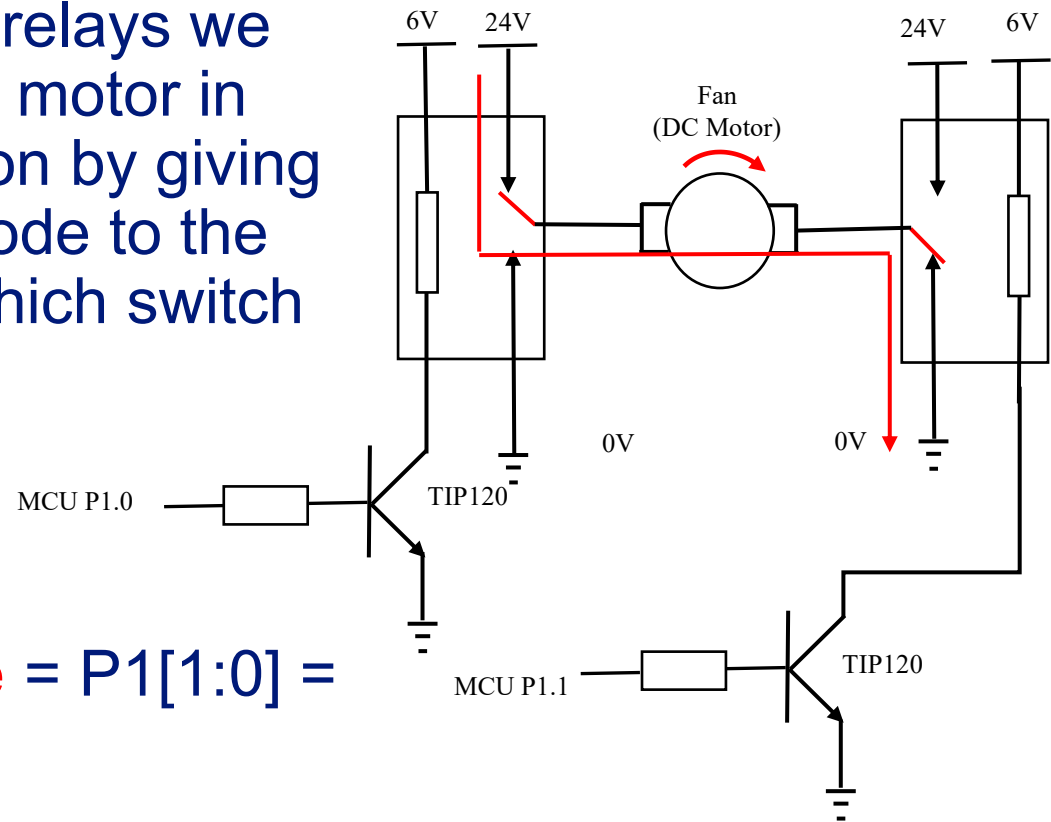
Motor off = $P[1:0] = 00$



H-Bridge Relay Circuit

Clockwise

- Using SPDT relays we can drive the motor in either direction by giving the correct code to the transistors which switch on the relay.



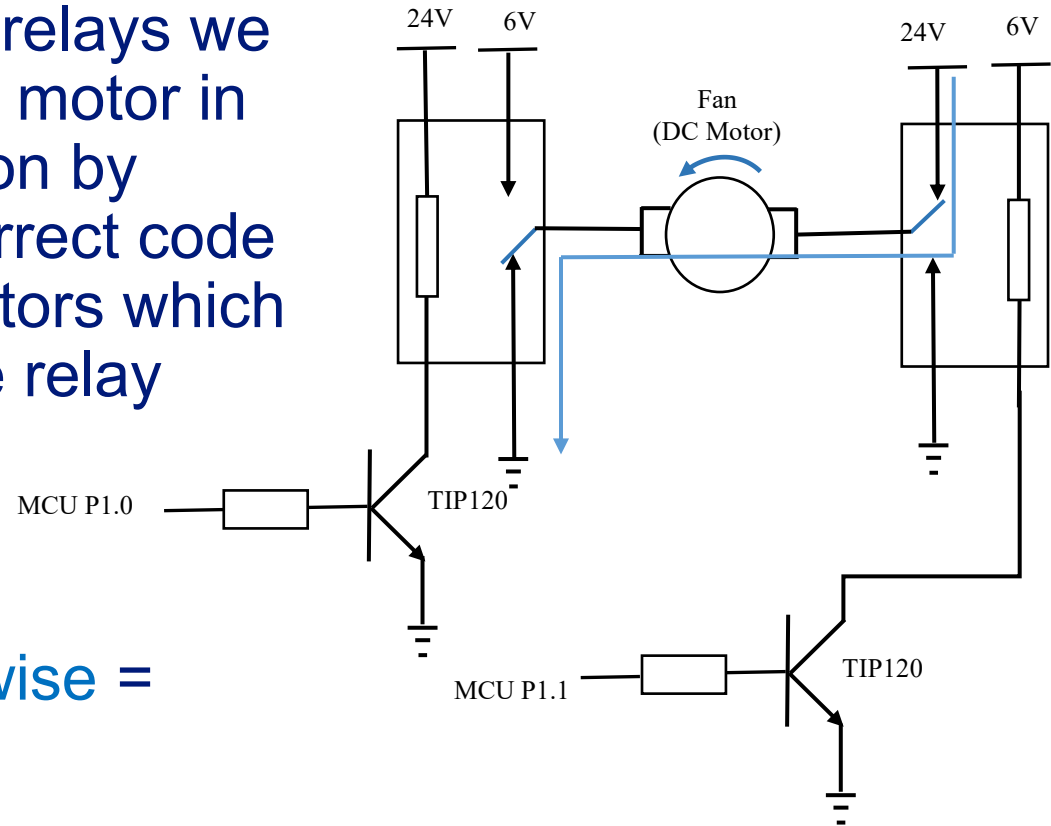
Signals:

Motor Clockwise = $P1[1:0] = 10$

H-Bridge Relay Circuit

Anticlockwise

- Using SPDT relays we can drive the motor in either direction by giving the correct code to the transistors which switch on the relay



Signals:

Motor Anticlockwise =
 $P1[1:0] = 01$

Interfacing to LCD Displays

Liquid Crystal Displays (LCDs)

- LCDs are available in two broad types:
 - I. Standard (Alphanumeric) Liquid Crystal Displays (LCDs) and
 - II. Graphical Liquid Crystal Displays (GLCDs)
- In this part of the module we take a look at interfacing a standard 16x2 LCD display with a microcontroller.



Introduction



- An LCD is a very useful component of any microcontroller system –
 - it allows the microcontroller to communicate meaningfully with the user.
- Several types/sizes but are usually grouped according to the number of (ASCII) characters they can display per line and the total number of possible displayed lines –
 - 8x1, 8x2, 16x1, 16x2, 20x1, 20x2, 40x4 etc are all available as LCD modules.
 - Note 16x2 implies 16 characters (including blank spaces!) can be displayed on each of 2 lines of display, etc.



Introduction

- Regardless of physical size, all LCD modules function in a broadly similar way.
- For our purposes we will use a 16x2 LCD display.

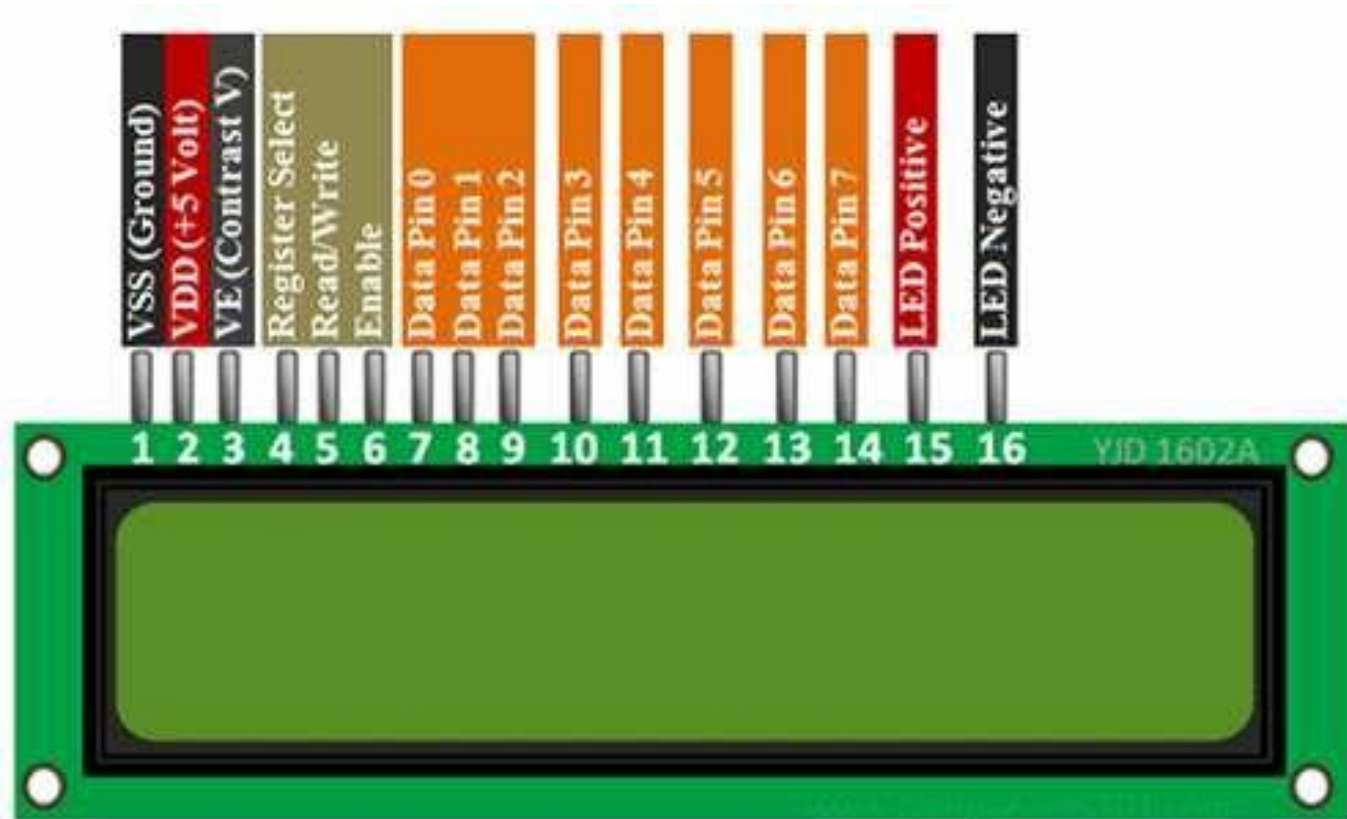


Hardware Pin Assignments

Industry Standard Pin Assignment for LCD Modules having a parallel interface and maximum of 80 Characters

Pin number	Symbol	Level	Function
1	Vss	-	- Power supply (GND)
2	Vcc	-	+ Power supply (+3.3V or 5V depending on LCD type)
3	Vo	Variable - Not used on some models	Contrast adjust
4	RS	0/1	0 = Instruction input 1 = Data input
5	R/W	0/1	0 = Write to LCD module 1 = Read from LCD module
6	E	0/1, Active:1->0	Enable signal
7	DB0	0/1	Data bus line 0 (LSB)
8	DB1	0/1	Data bus line 1
9	DB2	0/1	Data bus line 2
10	DB3	0/1	Data bus line 3
11	DB4	0/1	Data bus line 4
12	DB5	0/1	Data bus line 5
13	DB6	0/1	Data bus line 6
14	DB7	0/1	Data bus line 7 (MSB)
15	+Vcc	Only on some models	Backlight +Vcc, if available
16	Vss (0V)	Only on some models	Backlight GND, , if available

Standard pin Layout



Standard pin Layout

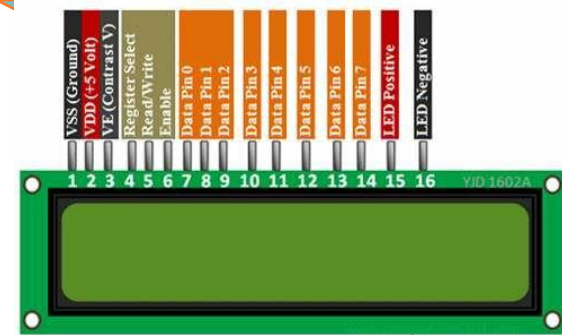
Pins 1 and 2 (VSS & VDD)

- The modules we will be using in the lab is a 3.3v module therefore:
 - Pin 1 -> GND
 - Pin 2 -> 3.3v

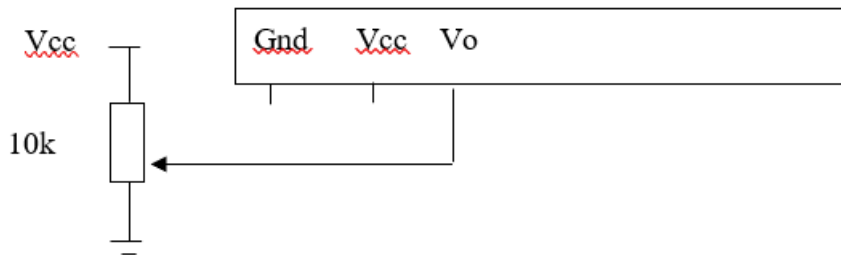


Standard pin Layout

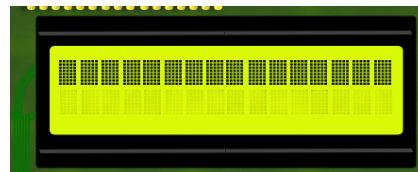
Pin 3 (Contrast Adjust)



- When an LCD module is powered up, very often nothing at all is visible on the display.
- The module's contrast must first be adjusted to suit the viewing conditions/viewing angle. This is achieved using a potentiometer connected to the contrast adjust pin as shown below;



- The pot. is adjusted until a line of solid bars similar to that shown below is visible on the LCD module. The LCD is then ready for further use



- NOTE: you can just connect Pin 3 to GND for max contrast.**

Control Lines

Pin 4 (RS)



- **Register Select (RS) Pin 4:**
- Two types of data may be written to the LCD display – Instruction (or format) data and actual Display (or message) data.
- **Instruction data is used to**
 - I. Initialise the LCD module at power up. Many initialisation options have to be decided e.g 1 or 2 line display, cursor ON/OFF/Blinking, text increment to left or right, etc
 - II. Send format instructions such as “carriage return” , (CR) when a new line is required, or “Clear Display” when an entirely new screen of messages is to be displayed
- All instruction data is sent with RS=0, meaning that the byte currently on the data lines is to be interpreted as a format instruction and not to be displayed.

Partial LCD Instruction set

Pin 4 RS Instruction Data: See LCD datasheet for full set

. Partial LCD instruction set

Instruction	Code		Description								Execution time**
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
Clear display	0	0	0	0	0	0	0	0	0	1	1.64mS
Cursor home	0	0	0	0	0	0	0	0	1	*	1.64mS
Entry mode set	0	0	0	0	0	0	0	1	I/D	S	40uS
Display On/Off control	0	0	0	0	0	0	1	D	C	B	40uS
Cursor/display shift	0	0	0	0	0	1	S/C	R/L	*	*	40uS
Function set	0	0	0	0	1	DL	N	F	*	*	40uS

Remarks:

DDRAM = Display Data RAM.

DDRAM address corresponds to cursor position.

- * = Don't care.

- ** = Based on $F_{osc} = 250kHz$.

Explanations for Table B. Bit names

Bit name Setting / Status

I/D	0 = Decrement cursor position	1 = Increment cursor position
S	0 = No display shift	1 = Display shift
D	0 = Display off	1 = Display on
C	0 = Cursor off	1 = Cursor on
B	0 = Cursor blink off	1 = Cursor blink on
S/C	0 = Move cursor	1 = Shift display
R/L	0 = Shift left	1 = Shift right
DL	0 = 4-bit interface	1 = 8-bit interface
N	0 = 1/8 or 1/11 Duty (1 line)	1 = 1/16 Duty (2 lines)
F	0 = 5x7 dots	1 = 5x10 dots
BF	0 = Can accept instruction	1 = Internal operation in progress

Pin 4 (RS): Display Data

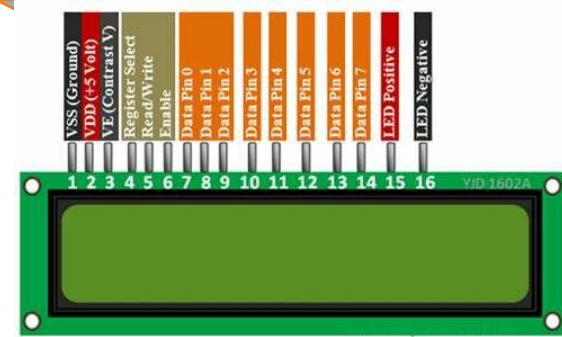
Least Significant. Nibble

Display Data: these are the actual (ASCII) characters which are to appear on the Display

All Display (message) data is sent with RS=1. meaning that the byte currently on the data lines is to be displayed at the current cursor position. Valid data bytes (ASCII codes which can be shown on the LCD) are given here.

Control Lines

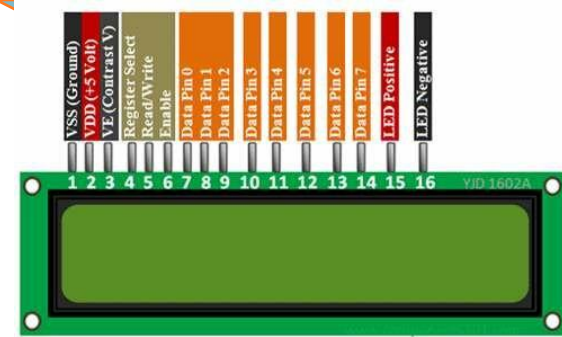
Pin 5 & 6 (R/W & EN)



- **Read/Write (R/W): Pin 5**
 - It is possible to both write to and read from a standard LCD module.
 - The R/W line determines whether a READ or a WRITE operation is in place.
 - R/W = 0 implies a WRITE operation (of data to LCD)
 - R/W = 1 implies a READ operation.
 - For our purposes, it will be unnecessary to read the LCD, thus we will connect R/W to GND and carry out only write operations.
- **Enable (EN): Pin 6**
 - Data (instruction or message) is accepted into the LCD module on the negative (falling) edge of the LCD Enable (EN) line.

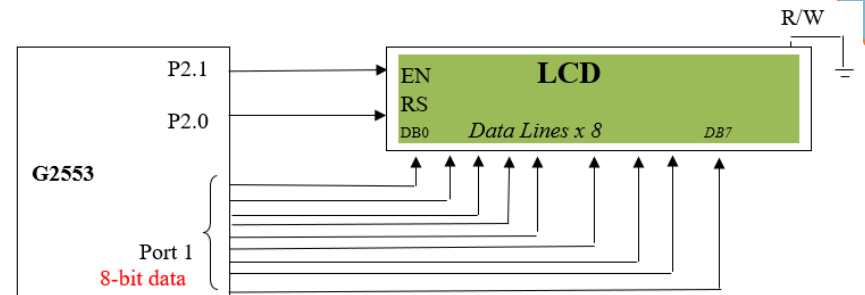
Data Lines (DB0 – DB7)

Pins 7-14 (8 bit mode)



- Two Configurations possible
 - 8-bit mode
 - and 4-bit mode

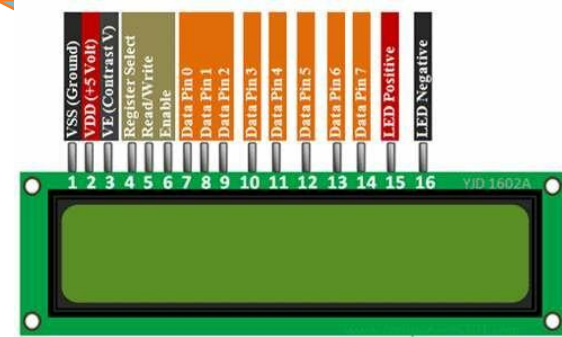
Notes:



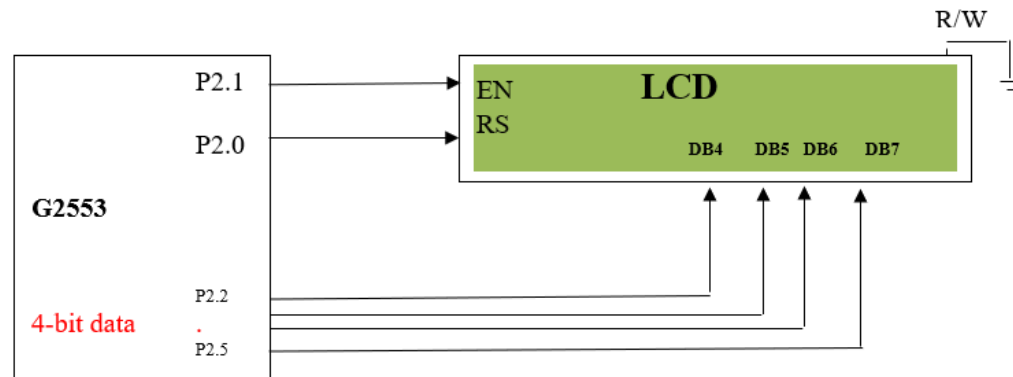
- data lines (8-bits, P1.0....P1.7, in this case) is connected to LCD data lines DB0-DB7 (also 8-bits). All data and instructions bytes are sent to LCD via these data lines. Port 2, in this case is used for control lines RS and EN.
- These port connections can be changed, if the user wishes
- In this mode, 8-bit data/instructions are sent on an 8-bit parallel bus (Port 1) to the 8-bit input DB0 – DB7 of LCD. Hence 8-bit mode
- R/W is tied to GND. Thus we can write to the LCD module but not read data back from it.
- EN (P2.1) and RS (P2.0) are used to control the flow of data to the LCD.
- The standard LCD connection uses 10 lines of the MSP430G2553. Most of the I/ O capability is used up by the LCD connection

Data Lines (DB0 – DB7)

Pins 11-14 (4 bit mode)



- The entire interface fits on 6 lines Port lines. Port 2 is used in this case. P2.2.....P2.5 (4-bits) are connected to LCD data lines DB4-DB7 (also 4-bits). All data and instructions bytes are sent to LCD these 4 lines. P2.0 and P2.1 are used for control lines RS and EN, as before These port connections can be changed, if the user wishes
- In this mode, 8-bit data/instructions are sent on a 4-bit parallel bus (Port 2.2 – P2.5) to the 4-bit input DB4 – DB7 of LCD. Hence 4-bit mode. To send an 8-bit byte over a 4-bit bus the byte is transmitted in two separate nibbles. The HIGH nibble is transmitted first, followed by the lower nibble
- R/W is tied to GND: Thus we can write to the LCD module but not read data back from it.
- EN (P2.1) and RS (P2.0) are used to control the flow of data to the LCD.
- The standard LCD connection uses 6 lines of the MSP430G2553., and is contained on less than one port



Sending 1 Byte of Instruction Data

1. Make LCD lines LOW (datax4, RS and EN)
2. Set RS LOW (byte of Instruction being sent to LCD)
3. Bring EN line HIGH
4. Place the instruction HiGH NIBBLE for the display on the data lines i.e. to P2.2...P2.5 Delay 1 ms
5. Bring EN line LOW, then and leave it in LOW for 2ms approx., with the instruction nibble still present on 4 data lines. This clocks the instruction into the LCD
6. Repeat steps 1-5 for the LOW NIBBLE of the instruction byte
7. Allow the LCD some time to process the received byte – the required amount of time varies according to the type of instruction received (see execution times on page 5, for examples) but never exceeds 2ms. Thus delay 2ms before any further communication with the LCD display.
8. Remember RW is connected LOW permanently (grounded)

Sending 1 Byte of Data

1. Make LCD lines LOW (datax4, RS and EN)
2. Set RS HIGH (byte of Data being sent to LCD)
3. Bring EN line HIGH
4. Place the data HiGH NIBBLE for the display on the data lines i.e. to P2.2...P2.5 Delay 1 ms
5. Bring EN line LOW, then and leave it in LOW for 2ms approx., with the instruction nibble still present on 4 data lines. This clocks the instruction into the LCD
6. Repeat steps 1-5 for the LOW NIBBLE of the data byte
7. Allow the LCD some time to process the received byte – the required amount of time varies according to the type of data received (see execution times on page 5, for examples) but never exceeds 2ms. Thus delay 2ms before any further communication with the LCD display.
8. Remember RW is connected LOW permanently (grounded)

What's the difference?

- Notice that the only real difference between writing instruction bytes and data bytes to the LCD is the state of the RS line while the transfer is taking place.

Summary

- Interfacing with mechanical switches
 - Problems – Switch Bounce
- Floating pins
 - Why its bad and how to avoid – Pull up/Pull Down resistors
- Interfacing with Matrix Keypads
- Interfacing to LEDs
- Interfacing with 7 Segment Displays
 - (Common Anode, Common Cathode)
 - BCD and Multiplexing
- Interfacing with DC Motors
 - Relays, H-bridges – Supplying enough current and direction control. Isolating Higher voltages from MCU
- Interfacing with LCD Displays
 - Pins, control signals, timings



